



**JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY**

SCHOOL OF OPEN, DISTANCE AND eLEARNING

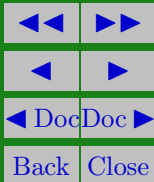
P.O. Box 62000, 00200

Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

ICS 2104 Object-Oriented Programming I

JKUAT SODEL
©2017





ICS 2104 Object-Oriented Programming I

This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 9

Template

Learning outcomes

Upon completing this topic you should be able to:

- Define template
- Differentiate between template and overloading
- Declare class template and function template
- Use templates

9.1. Introduction to Templates

Template is the foundation of generic programming. In generic programming, programmers define general classes and functions.



ICS 2104 Object-Oriented Programming I

template is a family of either classes or functions.

What is a template?

A template is a family of functions and classes. In creation of either a class or a function template, one writes only a single code from where other classes or functions are created.

9.2. Function Template

A function template is a family of functions. For example, if we want to swap integer, characters and floating points, we can overload swap() function as shown below.

```
//to swap integers  
int swap(int *a, int *b)  
{  
    int temp=*a;
```



ICS 2104 Object-Oriented Programming I

```
}
//to swap characters
char swap(char *a, char *b)
{
    char temp=*a;
    *a=*b;
    *b=temp;
}
//to value that have decimal points
double swap(double *a, double *b)
{
    double temp=*a;
```



```
}  
    *a=*b;  
    *b=temp;  
}
```

We have now defined three functions called swap. One function swaps values of type int, the other swaps values of type char while the other swaps values of type double. We have done that by overloading swap() function. However, the three functions swap the values in the same way. In such a case instead of writing these three functions, we could have written a function template and from this template we can create these three function. This is a powerful concepts. In that case we can create a swap function from our template that can be used to swap any data.

The syntax of declaring function template is:

JKUAT: Setting trends in higher Education, Research and Innovation



ICS 2104 Object-Oriented Programming I

template <class Type>

function-definition

Type is an identifier that is used to specify the following:

- a) Type of arguments that will be passed to the function.
- b) The return of the function
- c) To declare variables within the function

For example

```
template<class Type>
Type larger (Type x, Type y)
{
    if (x >= y)
        return x;
    else
        return y;
}
```



ICS 2104 Object-Oriented Programming I

```
}
```

where template and class are reserved words.

Now we have declared a set of functions that are passed two arguments and return the larger argument.

The types of parameters have not been specified.

This means we can pass to the larger() functions two arguments of the any data types.

Below is a program that shows how function templates are used.

```
/*  
template.cpp This program demonstrates  
how function template is used  
#include <iostream.h>
```




ICS 2104 Object-Oriented Programming I

```
//function template declaration  
template<class Type>  
Type larger(Type x, Type y);
```

```
int main(void)  
{  
    cout<<print the larger value <<larger(20,  
    cout<<print the larger value <<larger('A',  
    cout<<print the larger value <<larger(3.23  
    return 0;  
}  
template<class Type>  
Type larger(Type x, Type y)  
{
```



ICS 2104 Object-Oriented Programming I

```
    if (x >= y)
        return x;
    else
        return y;
}
```

If you compile and execute this program it will display the following on the screen:

```
print the larger value 30
print the larger value B
print the larger value 3.23
```

9.3. Class Template

Like function template, class template enable one to define a set of code that can be used to create several classes.



ICS 2104 Object-Oriented Programming I

The syntax for creating class template is

```
template<class Type>
```

class declaration

For example:

```
template<class Type>
```

```
class Number
```

```
{
```

```
    private:
```

```
        Type x;
```

```
    public:
```

```
        Number( Type );
```

```
        void print_value( void );
```

```
};
```

```
int main( void )
```



ICS 2104 Object-Oriented Programming I

```
{  
    Number<int> num1(20);  
    Number<char>num2('A');  
    Number<float>num3(3.14);  
    num1.print_value();  
    num2.print_value();  
    num3.print_value();  
    return 0;  
}  
  
//defining Number outside the class declaration  
template<class Type>  
Number<Type>r::Number(Type type)  
{  
    x=type;  
}
```



ICS 2104 Object-Oriented Programming I

```
}  
//defining print_value outside the class declarati  
template<class Type>  
void Number<Type>:: print_value(void)  
{  
    cout<<"The value is "<<x<<endl;  
}
```

If you compile and execute this program it will print the following on the screen

The value is 20 The value is A The value is 3.14



ICS 2104 Object-Oriented Programming I

Revision Questions

EXERCISE 1.  Suppose `cType` is a class template and `func` is a member function of `cType`. Show the heading of the function definition of `func`

Example . Suppose `cType` is a class template, which can take `int` as a parameter. Explain what the statement below means.

`cType<int> y;`

Solution: it declares `y` to be an object of the type `cType`, and the type passed to the class `cType` is `int` □

EXERCISE 2.  Find the error(s) in the following code.

```
template<class type>
class Strange
```



ICS 2104 Object-Oriented Programming I

```
{  
    ...  
};  
  
strange<int> s1;  
strange<type> s2;
```

EXERCISE 3. Consider the following declaration:

```
template<class type>  
class strange  
{  
    ...  
private:  
    type a;  
    type b;
```



ICS 2104 Object-Oriented Programming I

```
};
```

Write a statement that declares `sobj` to be an object of the type `strange` such that the private data members `a` and `b` are of type `int`

EXERCISE 4. Consider the definition of the following function template

```
template <class type>
type surprise(type x, type y)
{
    return x+y;
}
```

What is the output of the following statement

- a) `cout<<surprise(5, 7)<<endl;`
- b)



ICS 2104 Object-Oriented Programming I

```
string str1="sunny";  
string str2="day";  
cout<<surprise(str1, str2)<<endl;
```



References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)



ICS 2104 Object-Oriented Programming I

Solutions to Exercises

Exercise 1. `template<class Type> functionReturnType cType<Type>
func(parameters);`

Exercise 1